

BiteFixes Enterprise Architecture Manual

Volume 02

Engineering Principles & Standards

Document ID: BFA-002

Version: 1.0

Status: Draft

Language: English

Architecture Level: Enterprise

Copyright

BiteFixes SaaS Enterprise Platform

Enterprise Architecture Documentation

All rights reserved.

Revision History

Version	Date	Description
1.0	2026	Initial Engineering Principles

Table of Contents

1. Introduction
2. Architecture Philosophy
3. Engineering Principles
4. Design Principles
5. Domain Driven Design
6. Dependency Rules
7. Coding Standards
8. Folder Structure
9. Naming Standards
10. Entity Standards
11. API Standards
12. Database Standards
13. Security Standards
14. Logging Standards
15. Observability Standards
16. Testing Standards
17. AI Development Standards
18. Development Workflow

19. Architecture Review Checklist

20. Conclusion

Chapter 1

Introduction

Purpose

This document defines the official engineering standards for the BiteFixes Enterprise Platform.

Every developer, architect and contributor must follow these standards.

These standards ensure:

- Maintainability
- Scalability
- Readability
- Security
- Testability
- Long-term evolution

Chapter 2

Architecture Philosophy

The architecture follows one simple rule.

Business defines software.

Never the opposite.

The implementation order is always:

Business



Domains



Use Cases



APIs



Database



Implementation

Not:

Database



Python



Business

Chapter 3

Engineering Principles

Principle 1

Business First

Every feature starts with a business problem.

Never with code.

Principle 2

Domain Driven

Every feature belongs to one domain.

No shared business logic.

Principle 3

Single Responsibility

Each component has one responsibility.

Example

Correct

`CustomerService`

Incorrect

`CustomerTicketInventoryService`

Principle 4

Loose Coupling

Domains communicate through interfaces.

Never direct dependencies.

Ticket Domain

↓

ICustomerRepository

↓

Customer Domain

Principle 5

High Cohesion

Components that work together stay together.

Principle 6

Configuration Over Code

Everything configurable belongs in configuration.

Examples

- Business Hours
- Supported Languages

- Workflow Steps
- Notification Rules
- AI Parameters

Never hardcoded.

Principle 7

Security by Design

Security is never optional.

Every feature is secure by default.

Principle 8

Observability by Design

Every operation must be observable.

Metrics.

Logs.

Tracing.

Audit.

Principle 9

Event Driven

Important actions generate events.

Customer Registered

↓

CustomerRegisteredEvent

↓

Notification

Analytics

Audit

Bitey

Reports

Principle 10

AI Independent

AI does not implement business rules.

Business rules belong to the Business Core.

Chapter 4

Domain Driven Design

Each domain owns:

Entities

Value Objects

Repositories

Policies

Use Cases

Services

Events

DTOs

Validators

Tests

Nothing else.

Chapter 5

Dependency Rule

Dependencies always point inward.

Presentation

↓

Application

↓

Domain

↓

Infrastructure

Never:

Infrastructure

↓

Domain

Chapter 6

Folder Structure

Official project structure

app/

kernel/

business/

customer/

ticket/

device/

inventory/

knowledge/

ai/

`integrations/`

`infrastructure/`

`shared/`

`api/`

`tests/`

`docs/`

`scripts/`

Every module follows the same organization.

Chapter 7

Naming Standards

Python Files

Correct

`customer_service.py`

Incorrect

`customerServiceFinal.py`

Classes

CustomerService

TicketRepository

KnowledgeEngine

WorkflowManager

Interfaces

ICustomerRepository

ITicketRepository

IStorageProvider

DTO

CustomerDTO

TicketDTO

DeviceDTO

Events

CustomerRegisteredEvent

TicketClosedEvent

KnowledgePublishedEvent

Use Cases

RegisterCustomerUseCase

CloseTicketUseCase

GenerateEstimateUseCase

Chapter 8

Entity Standards

Every entity must contain

InternalID

UUID

CompanyID

Status

CreatedAt

UpdatedAt

CreatedBy

UpdatedBy

Version

Metadata

Mandatory.

Chapter 9

API Standards

Every API must be versioned.

Example

`/api/v1`

`/api/v2`

`/api/v3`

Never break compatibility.

Chapter 10

Database Standards

The database is not the business model.

The database stores business information.

Principles:

- Normalized
- Indexed
- Multi-tenant
- Soft Delete
- Audit Ready
- UUID Public IDs

- BIGINT Internal IDs

Chapter 11

Security Standards

Authentication

Authorization

Permissions

Roles

Encryption

Secrets

Audit Logs

JWT

Rate Limiting

HTTPS

Chapter 12

Logging Standards

Every operation logs:

Timestamp

User

Company

Module

Action

Execution Time

Result

Error

Chapter 13

Observability

Monitor:

API Response Time

Database Performance

AI Latency

Queue Size

Memory Usage

CPU Usage

Error Rate

Ticket Volume

Chapter 14

Testing

Every module must include

Unit Tests

Integration Tests

API Tests

Security Tests

Performance Tests

Chapter 15

AI Standards

Bitey follows:

Perception

↓

Understanding

↓

Memory

↓

Intent



Knowledge



Planning



Decision



Execution



Reflection



Learning

AI never manipulates the database directly.

Chapter 16

Development Workflow

Business Requirement



Architecture Review



Domain Design



Use Cases



API



Database



Implementation



Tests



Documentation



Deployment

Chapter 17

Architecture Review Checklist

Before implementing any feature answer:

- ✓ Which domain owns it?
- ✓ What business problem does it solve?
- ✓ Which events are generated?
- ✓ Which policies apply?
- ✓ Which APIs change?
- ✓ Which database objects change?
- ✓ Which AI capability uses it?
- ✓ Which tests validate it?

Chapter 18

Golden Rule

Every line of code must answer:

Does this improve the business model without breaking the architecture?

If the answer is **No**, redesign before implementing.

Conclusion

This document establishes the engineering foundation for BiteFixes Enterprise Platform.

Following these principles ensures that the platform remains:

- Maintainable
- Scalable
- Modular
- Secure
- Observable
- AI-Ready
- Enterprise-Grade